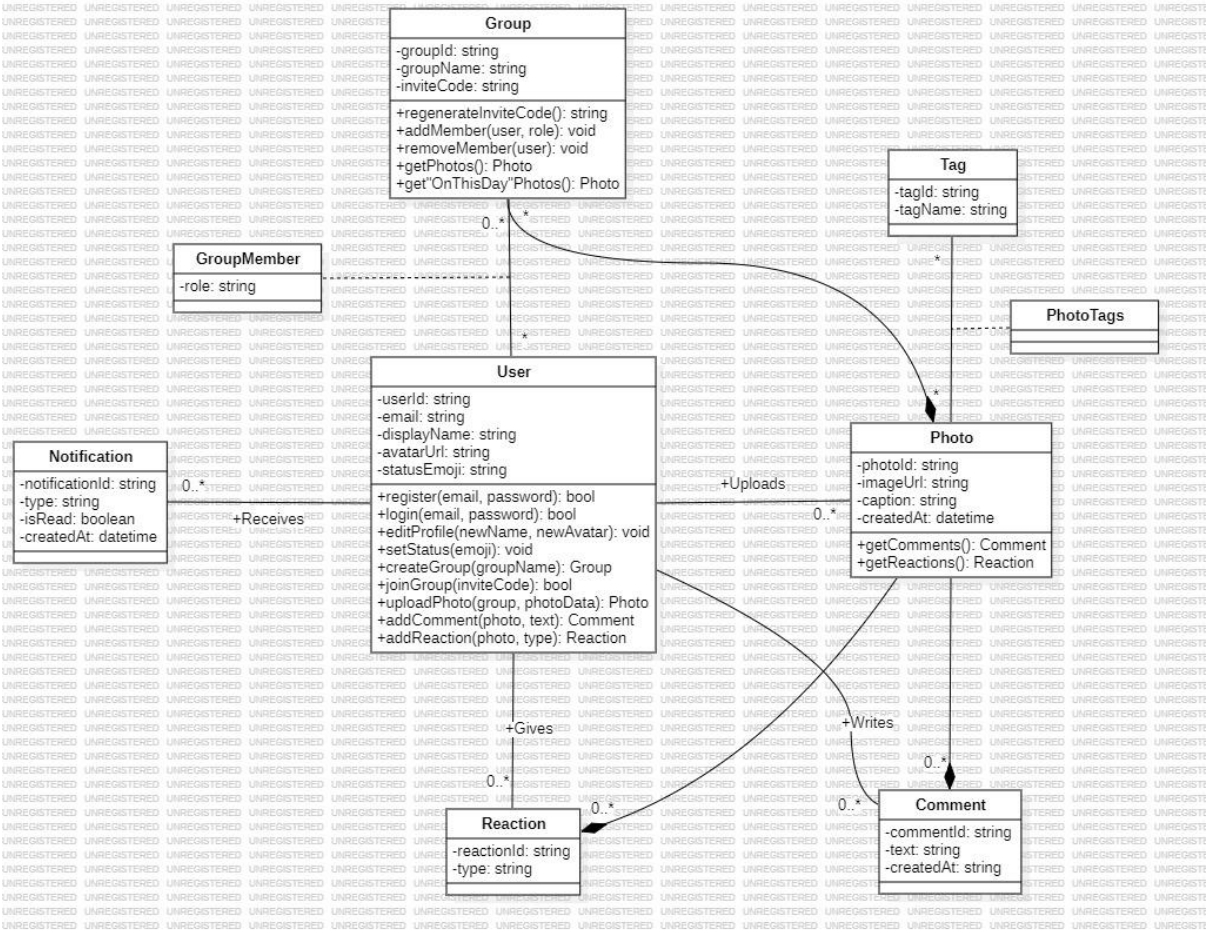
CLASS DIAGRAM

Name: G.Sasmith
RegNo: 23BPS1074



1. Class Diagram

The class diagram shown here provides a static, object-oriented view of the MomentsHub system. It serves as a technical blueprint, translating the project's data (from the ERD) and user actions (from the Use Cases) into a set of classes with defined attributes, operations, and relationships.

1.1 Core Classes and Responsibilities

The diagram is built around several key classes, each with a distinct responsibility:

- **User**: This class represents the system's actors. It stores state information like userId, email, and displayName. It also holds the primary user-initiated operations (methods), such as register(), login(), createGroup(), and addComment().
- **Group**: This class models the collaborative "capsules." It holds groupName and the inviteCode. Its operations, like regenerateInviteCode() and addMember(), are focused on the management of the group itself.
- **Photo**: This is the central content entity. It contains attributes for the imageUrl, caption, and createdAt timestamp. Its operations, getComments() and getReactions(), are designed to retrieve associated data.
- **Supporting Classes**: Comment, Reaction, Notification, and Tag are data-centric classes that model the other key entities required for the system's features.

1.2 Key Relationships and Multiplicity

The relationships between these classes define the system's structure and business rules:

1. **Composition (Strong Ownership):** This "owns-a" relationship (shown with a filled black diamond) indicates that the "part" class's lifecycle is dependent on the "whole" class.
    - **Group to Photo**: A Group is *composed of* many (0..*) Photos. This correctly models that if a group is deleted, all photos within it are also deleted.
    - **Photo to Comment/Reaction**: A Photo is *composed of* many (0..*) Comments and Reactions. When a photo is deleted, all its associated comments and reactions are deleted as well.
2. **Association Class (Many-to-Many):** This is used when a relationship between two classes has its own unique attributes.
    - **GroupMember**: This class links User and Group in a many-to-many (* to *) relationship. This is the correct way to model that

a user can be in many groups and a group can have many users. The GroupMember class itself stores the role (e.g., 'admin' or 'member') that a specific user has in a specific group.

- **PhotoTags**: This class similarly links Photo and Tag in a many-to-many relationship, allowing a photo to have multiple tags and a tag to be applied to multiple photos.

3. **Association (Interaction):** These simple "uses-a" links (shown with a plain line) define how classes interact.
    - **+Uploads**: A User can upload zero-to-many (0..*) Photos.
    - **+Writes**: A User can write zero-to-many (0..*) Comments.
    - **+Gives**: A User can give zero-to-many (0..*) Reactions.
    - **+Receives**: A User can receive zero-to-many (0..*) Notifications.

1.3 Conclusion

In summary, this class diagram provides a complete and formal model of the MomentsHub system's static structure. It clearly details the exact data (attributes) and behaviors (operations) each object will be responsible for and how they are all interconnected.